Enforce least-privilege access through identity-aware, application-specific access policies using Zscaler Private Access (ZPA).

List of teammates

S.no	Name	Collage	Contact
1.	ADITHYAN .S	Bishop Appasamy college of arts science	sadithyan368@gmail.com
2.	ABISHEK .A	Bishop Appasamy college of arts science	Abishek.ingo@gmail.com
3.	MESHACH FERNANDEZ .M	Bishop Appasamy college of arts science	meshahfernandez34@gmail.com
4.	MANAVALAN .M	Bishop Appasamy college of arts science	mmanavalan564@gmail.com
5.	SYED HAFNAS .A	Bishop Appasamy college of arts science	27syedhafnash@gmail.com

INTRODUCTION:

In an era where the digital landscape is constantly evolving, organizations face an ever-increasing threat from cyberattacks. The need for swift and effective incident response has become paramount to safeguard sensitive data, protect digital assets, and maintain the trust of stakeholders. In response to this growing challenge, we present a visionary project: the development of an AI-driven incident response platform that empowers cybersecurity teams with the tools to automate and enhance incident triage and response tasks.

In response to the evolving threat landscape and the dissolution of the traditional network perimeter, modern enterprises are rapidly shifting away from legacy VPN-centric remote access models. These conventional approaches often grant excessive network-level access, creating a significant attack surface and increasing the risk of lateral threat movement. To address these challenges, this project centers on the implementation of a Zero Trust security architecture, fundamentally operating on the principle of "never trust, always verify." By moving beyond location-based access, we aim to establish a security posture where trust is no longer implicit, and access is determined by granular context rather than network connectivity.

The core of this initiative involves the deployment of Zscaler Private Access (ZPA), a cloud-delivered Zero Trust Access solution designed to create identity-aware, application-specific connectivity. Unlike traditional VPNs that place users on the internal network, ZPA leverages a "service-initiated" model to establish direct, one-to-one connections between authorized users and specific applications, completely hiding the corporate network and IP address space from the internet. This ensures that users connect exclusively to the

applications they are permitted to access, based on continuous identity verification and context, rendering the applications invisible to unauthorized users and preventing common attack vectors such as port scanning and distributed denial-of-service attacks.

Ultimately, this project will enforce a least-privilege access model, ensuring users and devices are granted the minimum required access necessary to perform their functions. By decoupling application access from network access, we will eliminate the risk of lateral movement across the network, as users have no visibility or connectivity to other hosts. This strategic move not only significantly enhances security by reducing the attack surface but also simplifies the user experience and reduces operational overhead associated with legacy remote access infrastructure, positioning the organization at the forefront of secure and efficient access management.

ABSTRACT:
The proliferation of hybrid work and cloud-based resources has rendered traditional perimeter-based security models, such as VPNs, obsolete and increasingly dangerous. These legacy solutions often provide overly broad network access, exposing organizations to significant risks including lateral threat movement and an expanded attack surface. This project addresses these vulnerabilities by implementing a Zero Trust Architecture (ZTA) utilizing Zscaler Private Access (ZPA). The primary objective is to transition from network-centric to identity-centric access, thereby enforcing the principle of least privilege.

The methodology involves deploying ZPA's cloud-based service to create application-specific, identityaware micro-tunnels. Unlike conventional VPNs that place users on the internal network, this solution establishes direct, one-to-one connections between verified users and authorized applications. The network infrastructure and all non-authorized applications remain completely invisible to the end-user, effectively eliminating surface for reconnaissance and attack. Access policies are dynamically governed by user identity, device posture, and contextual attributes, ensuring that connectivity is strictly limited to what is explicitly required for specific job functions.

The implementation of ZPA successfully decouples application access from network access, resulting in a demonstrable reduction in the organization's attack surface and the complete prevention of lateral movement. This project delivers a robust, scalable, and user-friendly remote access framework that aligns with modern security best practices. By shifting to an identity-aware, least-privilege model, the organization significantly enhances its security posture while

simultaneously reducing operational complexity and improving the end-user experience.

Use Case Scenarios

Scenario 1: Finance Department Access to ERP System

A senior accountant in the finance department needs to access the corporate Enterprise Resource Planning (ERP) system to process monthly payroll. The user connects from a remote location using their company-managed laptop. Upon launching the ZPA Client Connector, they authenticate via Azure Active Directory with multi-factor authentication (MFA). ZPA evaluates the user's identity, group membership (Finance_ERP_Users), device posture (antivirus active, disk encryption enabled), and connection context. The policy engine determines that the user is authorized and brokers a secure, direct micro-tunnel to the ERP application only. The user can interact with the ERP interface, but has no visibility or network-level access to any other internal systems, such as development servers or the HR database.

Output:

• User authenticated and authorized based on identity and group membership
• Direct micro-tunnel established specifically to the ERP application
• All other internal applications and network segments remain invisible and inaccessible

Scenario 2: Contractor Access to Specific Project Repository

An external software consultant is hired for a three-month project and requires access only to a specific Git repository hosted on an internal server. The contractor is onboarded as a guest user in the identity provider and assigned to the "Project_X_Contractors" group. When the contractor connects using the ZPA client, the system verifies their identity and checks the device for basic security compliance (OS

patch level, firewall enabled). ZPA enforces a policy that grants access exclusively to the Git repository application (port 22/SSH or port 443/HTTPS) during business hours (9 AM to 5 PM). The contractor can clone, push, and pull code, but cannot discover or connect to any other internal resources, including the corporate intranet or file servers.

Output:

• Time-bound and role-specific access enforced
• Contractor restricted to a single application (Git repository)
• Lateral movement to adjacent systems completely blocked

Scenario 3: Executive Access to Confidential Board Documents

A company executive needs to access a confidential folder containing board meeting materials hosted on an internal file server. The executive connects from a tablet device while traveling internationally. ZPA integrates with the mobile device management (MDM) solution to verify that the tablet is compliant (not jailbroken, passcode enabled, encryption active). Upon successful authentication, ZPA evaluates the policy: "If user is in 'Executive_Board' group AND device is MDM-compliant AND connection is from a trusted country (allowlist), THEN grant access to 'Board_Documents' application." The executive is connected directly to the specific file share, and all data in transit is encrypted via TLS. Any attempt to access other shares or network resources is automatically blocked and logged.

Output:

• Context-aware access based on user role, device posture, and geographic location
• Granular access to a specific file share (Board_Documents)
• Comprehensive logging of access events for audit and compliance

Scenario 4:
IT Administrator Access to Server Management Interfaces A systems administrator requires access to management interfaces (SSH for Linux servers and RDP for Windows servers) to perform routine maintenance. Instead of granting VPN access to the entire management network, ZPA is configured to treat each server or management interface as a separate application segment. The administrator authenticates and is presented only with the specific servers they are authorized to manage based on their IT subgroup (e.g., "Linux_Admins" see only Linux servers; "Windows_Admins" see only Windows servers). When the admin connects via SSH to a Linux server, ZPA creates a direct micro-tunnel to that specific IP and port combination. The admin cannot scan the subnet, access other servers, or initiate connections to non-authorized ports.
Output:
• Micro-segmentation at the individual server/interface level • Role-based access controlling visibility of specific management consoles
• Elimination of network-level reconnaissance capabilities for admins

Scenario 5:
Mergers and Acquisitions Integration A company acquires a smaller firm and needs to provide the acquired company's HR team with temporary access to the parent company's benefits enrollment application. The acquired company's users are added as external collaborators in the identity provider with limited attributes. ZPA policies are created that specifically grant the "Acquired_HR" group access only to the "Benefits_App" (a web application hosted internally). The acquired users install the ZPA client, authenticate, and are connected directly to the benefits application. Because ZPA provides access without network connectivity, the parent company's

entire internal network remains completely hidden and protected. The acquired users cannot ping, scan, or access any other corporate resource. After six months, when the integration is complete and users are migrated to the parent identity system, the temporary policies are revoked.

Output:

• Secure, temporary access provided without network exposure
• Complete network invisibility protecting the parent company's infrastructure
• Simplified deprovisioning through policy revocation rather than complex network changes

Stage 1

Title of the project :

Enforce least-privilege access through identity-aware, application-specific access policies using Zscaler Private Access

Overview:

In response to the evolving threat landscape and the dissolution of the traditional network perimeter, modern enterprises are rapidly shifting away from legacy VPN-centric remote access models. These conventional approaches often grant excessive network-level access, creating a significant attack surface and increasing the risk of lateral threat movement. To address these challenges, this project centers on the implementation of a Zero Trust security architecture, fundamentally operating on the principle of "never trust, always verify." By moving beyond location-based access, we aim to establish a security posture where trust is no longer implicit, and access is determined by granular context rather than network connectivity. The core of this initiative involves the deployment of Zscaler Private Access (ZPA), a cloud-delivered Zero Trust Access solution designed to create

identity-aware, application-specific connectivity. Unlike traditional VPNs that place users on the internal network, ZPA leverages a "service-initiated" model to establish direct, one-to-one connections between authorized users and specific applications, completely hiding the corporate network and IP address space from the internet. This ensures that users connect exclusively to the applications they are permitted to access, based on continuous identity verification and context, rendering the applications invisible to unauthorized users and preventing common attack vectors such as port scanning and distributed denial-of-service (DDoS) attacks.

Ultimately, this project will enforce a least-privilege access model, ensuring users and devices are granted the minimum required access necessary to perform their functions. By decoupling application access from network access, we will eliminate the risk of lateral movement across the network, as users have no visibility or connectivity to other hosts. This strategic move not only significantly enhances security by reducing the attack surface but also simplifies the user experience and reduces operational overhead associated with legacy remote access infrastructure, positioning the organization at the forefront of secure and efficient access management.

List of Vulnerability Table

S.no	Category name	Vulnerability Name	CWE/CVE - No
1.	Broken Access Control	Relative Path Traversal	CWE-23
2.	Security Misconfiguration	Configuration	CWE-16
3.	Software Supply Chain Failures	Unimplemented or Unsupported Feature in UI	CWE-447
4.	Cryptographic Failures	Weak Encoding for Password	CWE-261
5.	Injection	Improper Input Validation	CWE-20

6.	Insecure Design	Permissive List of Allowed Inputs	CWE-183
7.	Authentication Failures	Improper Authentication	CWE-287
8.	Software or Data Integrity Failures	Untrusted Search Path	CWE-426
9.	Security Logging & Alerting Failures	Information Loss or Omission	CWE-221
10.	Mishandling of Exceptional Conditions	NULL Pointer Dereference	CWE-476

REPORT:-

1.

Vulnerability Name:- Relative Path Traversal

CWE/CVE : -CWE-23

OWASP/SANS Category:-A01:2025 Broken Access Control

Description:-

Relative Path Traversal is a security vulnerability that occurs when an application allows users to access files and directories outside the intended directory structure by manipulating file path inputs. This vulnerability arises when a program does not properly validate or sanitize user-supplied file paths, enabling attackers to traverse through the system's directory structure and access restricted files.

Relative path traversal is commonly seen in web applications where file names or paths are taken from user input. Attackers exploit this vulnerability by inserting special directory navigation characters such as `../` into file paths. These characters instruct the system to move up one directory level, allowing attackers to access files located outside the permitted directory.

This vulnerability is documented in the **Common Weakness Enumeration as **CWE-23 and is often referred to as directory traversal.

Business Impact::-

1. Exposure of Sensitive Business Data

One of the most serious impacts is the unauthorized access to confidential files. Attackers may retrieve important business data such as:

- Customer information
- Financial records
- Internal documents
- Configuration files
- Source code

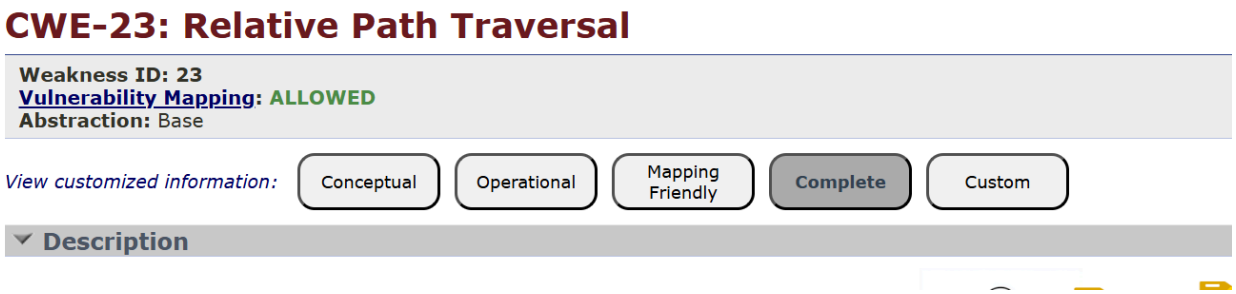
If this data is leaked, it can damage the organization's trust with customers and partners.

2. Financial Loss

Data breaches resulting from path traversal vulnerabilities can lead to significant financial losses. Organizations may face costs related to:

- Incident response and forensic investigations
- System recovery and remediation
- Legal fees and regulatory penalties
- Customer compensation

In some industries, data protection regulations require organizations to pay large fines when sensitive information is exposed.



2.

Vulnerability Name:- Configuration

CWE/CVE :-CWE-16

OWASP/SANS Category:-A02:2025 Security Misconfiguration

Description:-

A Configuration Vulnerability occurs when a system, application, or network is improperly configured, leading to potential security weaknesses that attackers can exploit. Configuration settings determine how software, servers, databases, and security mechanisms operate. If these settings are not correctly defined or maintained, they may expose sensitive information, allow unauthorized access, or weaken system defenses.

Many configuration vulnerabilities arise because systems are deployed with default settings, unnecessary services, or weak security controls. During installation, software often includes default configurations designed for ease of setup rather than strong security. If administrators fail to modify these settings, attackers may exploit them to compromise the system.

Configuration vulnerabilities are widely recognized in cybersecurity frameworks such as the **OWASP security guidelines, particularly under

the category **Security Misconfiguration. These vulnerabilities are among the most common causes of data breaches and system compromises.

Business Impact::-

1. Data Breaches and Information Exposure

Improper configuration can expose sensitive data such as:

- Customer personal information
- Financial records
- Business documents
- System configuration files

If attackers gain access to these resources due to weak configuration settings, it can result in data breaches, which may damage the organization's credibility and customer trust.

2. Financial Loss

Configuration vulnerabilities can lead to direct and indirect financial losses, including:

- Costs for incident investigation and response
- Expenses for fixing and upgrading systems
- Legal penalties and regulatory fines

- Compensation to affected customers

Organizations may also suffer revenue loss if services are interrupted due to security incidents.

CWE CATEGORY: Configuration

Category ID: 16
Vulnerability Mapping: PROHIBITED

Summary

Weaknesses in this category are typically introduced during the configuration of the software.

Membership

Nature	Type	ID	Name
MemberOf	V	635	Weaknesses Originally Used by NVD from 2008 to 2016
MemberOf	C	933	OWASP Top Ten 2013 Category A5 - Security Misconfiguration
MemberOf	C	1032	OWASP Top Ten 2017 Category A6 - Security Misconfiguration
MemberOf	C	1349	OWASP Top Ten 2021 Category A05:2021 - Security Misconfiguration
MemberOf	C	1437	OWASP Top Ten 2025 Category A02:2025 - Security Misconfiguration

Vulnerability Mapping Notes

Usage: PROHIBITED *(this CWE ID must not be used to map to real-world vulnerabilities)*

Reason: Category

Rationale:
This entry is a Category. Using categories for mapping has been discouraged since 2019. Categories are informal organizational groupings of weaknesses that can help CWE users with data aggregation, navigation, and browsing. However, they are not weaknesses in themselves [REF-1287]. This CWE entry may have become widely-used because of NIST's usage in NVD from 2008 to 2016 (see

3.

Vulnerability Name:- Unimplemented or Unsupported Feature in UI

CWE/CVE :-CWE-447

OWASP/SANS Category:-A03:2025 Software Supply Chain Failure

Description:-

Unimplemented or Unsupported Feature in UI is a software vulnerability that occurs when a user interface (UI) includes features, options, or controls that appear functional but are not actually implemented or supported by the underlying system. These incomplete features may mislead users into believing that certain operations are available or

properly secured, when in reality they are either non-functional or partially implemented.

This issue often arises during the software development process when UI elements such as buttons, links, or configuration options are added to the interface but the corresponding backend functionality is not developed or properly integrated. As a result, users may interact with these elements expecting them to perform a task, but the system may ignore the request, return an error, or behave unpredictably.

Business Impact::-

1. Misleading Security Controls

If a UI shows security-related features (such as enabling encryption, logging, or multi-factor authentication) that are not actually implemented, users or administrators may believe the system is secure when it is not. This **false sense of security** can expose the organization to serious cyber risks.

2. Poor User Experience

Users rely on the interface to perform tasks efficiently. If a feature appears available but does not work properly, it can lead to:

- User frustration
- Confusion about system capabilities
- Reduced productivity

This may negatively affect how customers or employees perceive the application.

CWE-447: Unimplemented or Unsupported Feature in UI

Weakness ID: 447
Vulnerability Mapping: ALLOWED
Abstraction: Base

View customized information: Conceptual | Operational | Mapping Friendly | **Complete** | Custom

Description

A UI function for a security feature appears to be supported and gives feedback to the user that suggests that it is supported, but the underlying functionality is not implemented.

Common Consequences

Impact	Details
Varies by Context; Unexpected State	**Scope: Other**

4.

Vulnerability Name:- Weak Encoding for Password

CWE/CVE :-CWE-261

OWASP/SANS Category:-A04:2025 Cryptographic Failures

Description:-

Weak Encoding for Password is a security vulnerability that occurs when passwords are stored or transmitted using weak or reversible encoding methods instead of strong cryptographic protection. Encoding techniques

are primarily designed to convert data into another format for transmission or storage, but they do not provide real security. If passwords are encoded using weak methods, attackers may easily decode them and gain unauthorized access to user accounts.

In secure systems, passwords should never be stored in plain text or using simple encoding mechanisms. Instead, they should be protected using strong cryptographic hashing algorithms that make it extremely difficult to recover the original password. When weak encoding methods are used, attackers who gain access to the system database can easily retrieve and decode the stored passwords.

This vulnerability is documented in the **Common Weakness Enumeration as **CWE-261 (Weak Encoding for Password).

How Weak Password Encoding Occurs

Weak encoding for passwords typically occurs due to poor security practices during application development. Some common causes include:

1. Using simple encoding methods such as Base64 instead of encryption or hashing.

2. Storing passwords in plain text within databases or configuration files.

3. Using outdated hashing algorithms that are easily cracked.

4. Failing to use salt values when hashing passwords.

Encoding methods like Base64 can be reversed easily, meaning anyone who gains access to the encoded password can quickly decode it back into the original password.

Business Impact::-

1. Unauthorized Access to Systems

If passwords are weakly encoded, attackers who obtain the stored password data can easily decode it and gain access to user accounts. This may allow attackers to access sensitive systems such as:

- Customer accounts
- Internal company systems
- Administrative dashboards
- Financial platforms

Unauthorized access can lead to further attacks and data manipulation.

2. Data Breaches

Weak password encoding can expose sensitive information stored in company databases. If attackers successfully decode user passwords, they may access confidential data such as:

- Personal customer information
- Payment details
- Business records

- Proprietary data

Data breaches can have severe consequences for both customers and the organization.

CWE-261: Weak Encoding for Password

Weakness ID: 261
Vulnerability Mapping: ALLOWED
Abstraction: Base

View customized information: Conceptual | Operational | Mapping Friendly | **Complete** | Custom

Description

Obscuring a password with a trivial encoding does not protect the password.

Extended Description

Password management issues occur when a password is stored in plaintext in an application's properties or configuration file. A programmer can attempt to remedy the password management problem by obscuring the password with an encoding function, such as base 64 encoding, but this effort does not adequately protect the password.

5.

Vulnerability Name:- Improper Input Validation

CWE/CVE :-CWE-20

OWASP/SANS Category:-A05:2025 Injection

Description:-

Improper Input Validation is a security vulnerability that occurs when a software application does not properly verify or sanitize the data provided by users or external sources before processing it. Input validation is a crucial step in secure software development because applications often

receive data from various sources such as user forms, APIs, files, and network requests. If this data is not carefully checked, attackers may inject malicious input that can manipulate the system or compromise security.

This vulnerability is categorized in the **Common Weakness Enumeration as **CWE-20. Improper input validation is also closely related to several common web security issues highlighted by **OWASP in its security guidelines.

Improper input validation occurs when an application fails to check whether the input data is in the correct format, type, length, or range before using it in operations such as database queries, file processing, or command execution. When this happens, attackers can supply unexpected or malicious data that may cause the program to behave incorrectly.

For example, if a web application expects a numeric value for a user ID but does not verify the input, an attacker might provide a string containing malicious commands instead of a number. This could lead to serious vulnerabilities such as SQL injection, command injection, or cross-site scripting (XSS) attacks.

Business Impact::-

1. Data Breaches and Information Exposure

Improper input validation can allow attackers to perform attacks such as SQL injection, command injection, or cross-site scripting. These attacks may allow unauthorized access to sensitive data such as:

- Customer personal information
- Financial data
- Business records

- Login credentials

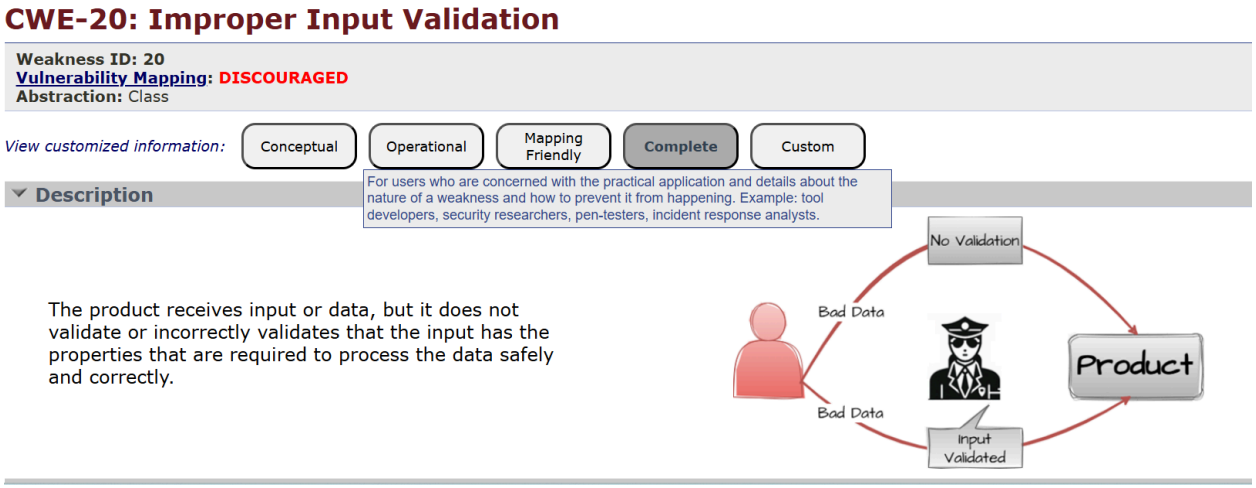
Exposure of such information can lead to major data breaches.

2. Financial Loss

Security incidents caused by improper input validation can result in significant financial costs, including:

- Expenses for investigating and fixing security issues
- System recovery and security upgrades
- Legal penalties and regulatory fines
- Compensation to affected customers

Organizations may also lose revenue if services are disrupted during the recovery process.



6.

Vulnerability Name:- Permissive List of Allowed Inputs

CWE/CVE :-CWE-183

OWASP/SANS Category:-A06:2025 Insecure Design

Description:-

Permissive List of Allowed Inputs is a security vulnerability that occurs when an application allows too many or overly broad input values without proper restrictions or validation. Instead of strictly defining what inputs are acceptable, the system permits a wide range of values, which may include malicious or unexpected data. This can create opportunities for attackers to exploit the application.

In secure software development, applications should validate user inputs carefully by allowing only specific, trusted values. However, when the list of allowed inputs is too permissive, the validation mechanism becomes weak and ineffective. As a result, attackers may inject harmful data into the system, potentially leading to security issues such as code injection, unauthorized access, or system malfunction.

This vulnerability is related to input validation weaknesses described in the **Common Weakness Enumeration database, particularly **CWE-183 (Permissive List of Allowed Inputs).

Business Impact::-

1. Increased Risk of Security Attacks

A permissive input validation system may allow attackers to inject malicious data into the application. This can lead to various attacks such as:

- SQL injection
- Command injection
- Cross-site scripting (XSS)
- Malicious file uploads

These attacks may compromise the security of the system and expose sensitive information.

2. Exposure of Sensitive Business Data

If attackers exploit weak input validation, they may gain unauthorized access to sensitive data stored in the system, such as:

- Customer personal information
- Financial records
- Business documents
- Login credentials

CWE-183: Permissive List of Allowed Inputs

Weakness ID: 183
Vulnerability Mapping: ALLOWED
Abstraction: Base

View customized information: Conceptual | Operational | Mapping Friendly | **Complete** | Custom

Description

The product implements a protection mechanism that relies on a list of inputs (or properties of inputs) that are explicitly allowed by policy because the inputs are assumed to be safe, but the list is too permissive - that is, it allows an input that is unsafe, leading to resultant weaknesses.

Alternate Terms

Allowlist / Allow List	This is used by CWE and CAPEC instead of other commonly-used terms. Its counterpart is denylist.
Safelist / Safe List	This is often used by security tools such as firewalls, email or web gateways, proxies, etc.
Whitelist / White List	This term is frequently used, but usage has been declining as organizations have started to adopt other terms.

7.

Vulnerability Name:- Improper Authentication

CWE/CVE :-CWE-287

OWASP/SANS Category:-A07:2025 Authentication Failures

Description:-

Improper Authentication is a security vulnerability that occurs when a system fails to correctly verify the identity of a user, device, or process before granting access to resources or functionality. Authentication is a critical security mechanism used to ensure that only authorized users can access protected systems, applications, or data. When authentication is implemented incorrectly or incompletely, unauthorized users may gain access to sensitive information or system functions.

This vulnerability is commonly associated with weak authentication mechanisms such as missing authentication checks, weak password validation, improper session handling, or flawed login processes. As a result, attackers may bypass authentication controls and access restricted areas of an application without valid credentials.

Business Impact::-

1. Financial Loss

- Attackers may access financial systems or payment information.
- Fraudulent transactions or theft of funds can occur.
- Businesses may also face recovery and investigation costs.

2. Data Breaches

- Unauthorized users can access sensitive data such as customer details, trade secrets, or internal documents.
- This can lead to large-scale data leaks.

3. Reputation Damage

- Customers lose trust if their accounts or data are compromised.
- Negative publicity can reduce customer loyalty and future sales.

4. Legal and Regulatory Penalties

- Companies may face fines if they fail to protect user data.
- Regulations like General Data Protection Regulation (GDPR) or California Consumer Privacy Act (CCPA) require strong data protection.

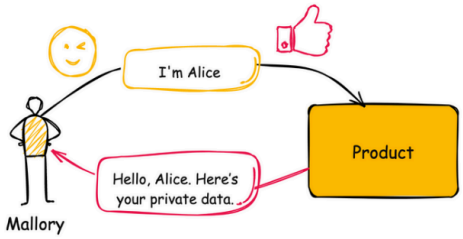
CWE-287: Improper Authentication

Weakness ID: 287
Vulnerability Mapping: DISCOURAGED
Abstraction: Class

View customized information: Conceptual | Operational | Mapping Friendly | **Complete** | Custom

Description

When an actor claims to have a given identity, the product does not prove or insufficiently proves that the claim is correct.



8.

Vulnerability Name:- Untrusted Search Path

CWE/CVE :-CWE-426

OWASP/SANS Category:-A08:2025 Software or Data Integrity Failures

Description:-

Untrusted Search Path is a software security vulnerability that occurs when a program searches for executable files or libraries in directories that may contain malicious or untrusted files. If an attacker places a malicious file in one of these directories, the application might unintentionally execute that file instead of the legitimate one. This can allow attackers to run unauthorized code within the system.

This vulnerability commonly occurs in operating systems such as Microsoft Windows and Linux, where applications search multiple directories to locate executable programs or dynamic libraries. The system follows a search path order, checking directories one by one until it finds the required file. If a malicious file is found earlier in the search path, it may be executed instead of the intended program.

For example, suppose an application needs to execute a system utility but does not specify the full path to the executable. The operating system will search through directories such as the current working directory, system directories, and other paths defined in environment variables. If an attacker places a malicious executable with the same name in a directory that is searched earlier, the program may execute the malicious file.

Business Impact::-

1. Unauthorized Execution of Malicious Code

If attackers exploit an untrusted search path vulnerability, they may place malicious executables or libraries in directories that the system checks first. When the application runs, it may execute the malicious file instead of the intended program. This can allow attackers to gain control over the system and perform unauthorized actions.

2. Data Breaches and Information Theft

Once attackers successfully execute malicious code through this vulnerability, they may access sensitive information stored in the system, such as:

- Customer data
- Financial records
- Internal business documents
- System credentials

Exposure of sensitive data can lead to serious security incidents and loss of confidential information.

CWE-426: Untrusted Search Path

Weakness ID: 426
Vulnerability Mapping: ALLOWED (with careful review of mapping notes)
Abstraction: Base

View customized information: Conceptual | Operational | Mapping Friendly | Complete | Custom

Description

The product searches for critical resources using an externally-supplied search path that can point to resources that are not under the product's direct control.

Extended Description

This might allow attackers to execute their own programs, access unauthorized data files, or modify configuration in unexpected ways. If the product uses a search path to locate critical resources such as programs, then an attacker could modify that search path to point to a malicious program, which the targeted product would then execute. The problem extends to any type of critical resource that the product trusts.

Some of the most common variants of untrusted search path are:

- In various UNIX and Linux-based systems, the PATH environment variable may be consulted to locate executable programs, and LD_PRELOAD may be used to locate a separate library.
- In various Microsoft-based systems, the PATH environment variable is consulted to locate a DLL, if the DLL is not found in other paths that appear earlier in the search order.

9.

Vulnerability Name:- Information Loss or Omission

CWE/CVE :-CWE-221

OWASP/SANS Category:-A09:2025 Security Logging & Alerting Failures

Description:-

Information Loss or Omission is a type of software vulnerability that occurs when a system fails to provide important information or loses critical data during processing, storage, or transmission. This issue can affect the reliability, security, and usability of an application because users or systems may not receive the necessary information required for proper decision-making or system operation.

In many software systems, applications are responsible for collecting, processing, and delivering information accurately. When some information is missing, incorrectly removed, or not transmitted, it may lead to incorrect results, incomplete records, or system errors. Information loss or omission can happen due to programming errors, improper data handling, insufficient validation, or system failures.

One of the major concerns with this vulnerability is that it can hide important events or security-related data, making it difficult for administrators to detect problems or security incidents. For example, if a logging system fails to record certain user activities or system errors, administrators may not be able to track suspicious behavior or diagnose issues effectively.

Information loss or omission can occur in several situations, including data transmission failures, incomplete data storage, incorrect database queries, or poor error handling in applications. For instance, when a program processes user input but fails to store some of the required fields in the database, the stored record becomes incomplete. Similarly, if a system transmits only partial data to another service due to a communication error, important information may be omitted.

Business Impact::-

1. Incorrect Business Decisions

Organizations rely on accurate data to make important business decisions. If information is lost or omitted from reports, databases, or system logs, management may make decisions based on incomplete or incorrect information. This can lead to poor planning, incorrect financial analysis, and ineffective strategies.

2. Data Integrity Issues

Information loss reduces the reliability and integrity of stored data. When data is incomplete or inconsistent, systems may produce inaccurate outputs or reports. This can affect business processes that depend on accurate records, such as financial management, inventory tracking, or customer service.

CWE-221: Information Loss or Omission

Weakness ID: 221
Vulnerability Mapping: ALLOWED (with careful review of mapping notes)
Abstraction: Class

View customized information: Conceptual | Operational | Mapping Friendly | Complete | Custom

Description

The product does not record, or improperly records, security-relevant information that leads to an incorrect decision or hampers later analysis.

Common Consequences

Impact	Details
Hide Activities	**Scope: Non-Repudiation**

10.

Vulnerability Name:- NULL Pointer Dereference

CWE/CVE :-CWE-476

OWASP/SANS Category:-A10:2025 Mishandling of Exceptional Conditions

Description:-

A Null Pointer Dereference is a common software vulnerability that occurs when a program attempts to access or modify memory using a pointer that has a NULL value. In programming languages such as C and C++, a pointer is a variable that stores the memory address of another variable. When a pointer is assigned a NULL value, it means that the pointer does not refer to any valid memory location.

If a program tries to dereference (access the value at the memory location of) a NULL pointer, it results in a runtime error because the program attempts to access memory that does not exist or is not allocated. This typically leads to program crashes, unexpected behavior, or security vulnerabilities.

How NULL Pointer Dereference Occurs

A NULL pointer dereference occurs when:

1. A pointer variable is declared but not initialized properly.
2. The pointer is explicitly assigned NULL but later used without validation.
3. Memory allocation fails and the program does not check if the allocation returned NULL.
4. Objects or resources are freed and later accessed again.

When the program tries to read or write data through this invalid pointer, the operating system usually terminates the program to prevent memory corruption.

Business Impact::-

1. Application Crashes and Service Downtime

One of the most common consequences of a NULL pointer dereference is application failure or system crash. When the program attempts to access a null memory location, the operating system may terminate the process to protect system stability.

This can lead to:

- Service interruptions
- System downtime
- Reduced availability of business applications

For organizations that rely on online services, downtime can directly impact business operations.

2. Loss of Productivity

When systems crash frequently due to programming errors like NULL pointer dereference, employees may not be able to complete their tasks efficiently. This can slow down business processes and reduce overall productivity.

IT teams may also spend additional time diagnosing and fixing such issues instead of focusing on other important tasks.

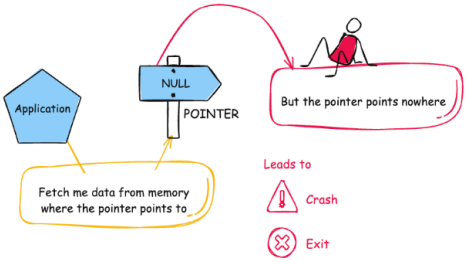
CWE-476: NULL Pointer Dereference

Weakness ID: 476
Vulnerability Mapping: ALLOWED
Abstraction: Base

View customized information: Conceptual | Operational | Mapping Friendly | **Complete** | Custom

Description

The product dereferences a pointer that it expects to be valid but is NULL.



Stage 2

Overview :-

An overview of a Nessus scan typically involves the following key steps and aspects:

1. Preparation: Define Objectives: Determine the scope and objectives of the scan. What systems or networks do you want to assess, and what specific vulnerabilities are you looking for? System Access: Ensure that you have the necessary permissions and access rights to scan the target systems or networks.

2. Configuration: Set Up Scan Policy: Create a scan policy that defines the parameters of the scan, such as the types of vulnerabilities to check for, scanning frequency, and specific configurations. Define Target Assets: Specify the IP addresses, hostnames, or network ranges of the systems you want to scan.

3. Scanning: Initiate Scan: Start the Nessus scan using the configured policy and target assets. Nessus will systematically scan the target systems for vulnerabilities based on its plugins and policies.

4. Vulnerability Identification: Nessus performs a comprehensive assessment, checking for vulnerabilities, misconfigurations, open ports, and other security issues. As the scan progresses, it categorizes vulnerabilities by severity, making it easier to prioritize remediation efforts.

5. Reporting: Nessus generates detailed reports that list the vulnerabilities discovered during the scan. Reports typically include information about the severity of each vulnerability, recommended actions for remediation, and supporting evidence. These reports can be customized and exported in various formats, such as PDF, HTML, or CSV.

6. Remediation: Review Findings: IT and security teams review the Nessus scan reports to prioritize and address the identified vulnerabilities. Remediation Steps: Take corrective actions to fix or mitigate the vulnerabilities. This may involve applying patches, changing configurations, or implementing security measures.

7. Follow-up: Regular Scanning: Conduct periodic Nessus scans to continuously monitor the security posture of the systems and networks. This helps identify new vulnerabilities and track progress in mitigating existing ones. Integration: Integrate Nessus with other security tools or workflows to automate the vulnerability management process and streamline the response to new findings.
8. Compliance Auditing: If compliance checks are part of your scan policy, review compliance results and take actions to ensure systems meet the required standards and regulations.

Nessus scans are a critical component of proactive security, helping organizations identify and address vulnerabilities in their IT infrastructure to reduce the risk of security breaches and data compromises. The frequency and depth of scans can vary depending on an organization's security strategy and requirements.

Target website - https://public-firing-range.appspot.com
Target ip address -142.251.214.148

List of vulnerability

s.no	Vulnerability name	Severity	plugins
1.	Service detection	Info	22964
2.	OS Fingerprints Detected	Info	
3.	Nessus SYN Scanner	Info	11219
4.	Device Type	Info	54615

5.	OS Identification	Info	11936
6.	Open SSL Detection	Info	50845
7.	Traceroute Information	Info	10287
8.	Shor's Harvest Now Decrypt Later	Info	298387

REPORT:-

1.

Vulnerability Name:-Service detection

severity : Info

Plugin:-22964

Port :-80 / tcp / www

Description:-Nessus was able to identify the remote service by its banner or by looking at the error message it sends when it receives an HTTP request.

Solution:-

1. Disable Unnecessary Services

2. Hide Service Banners (Banner Grabbing Protection)

3. Patch and Update Services

Business Impact::-

1. Service Downtime

- Critical systems stop working.

- Example: Company website or payment system becomes unavailable.

2. Data Breach

- Attackers access sensitive data (customer info, financial records).

3. Financial Loss

- Loss of revenue if services stop.

- Costs for incident response and recovery.

2.

Vulnerability Name:-OS Fingerprints Detected

severity : Info

Plugin:-209654

Description:-Using a combination of remote probes (TCP/IP, SMB, HTTP, NTP, SNMP, etc), it was possible to gather one or more fingerprints from the remote system. While the highest-confidence result was reported in plugin 11936, “OS Identification”, the complete set of fingerprints detected are reported here.

Solution:-

1. Enable Firewall Rules

2. Disable Unnecessary Services

3. Patch and Update the OS

Business Impact::-

1. Targeted Cyber Attacks

   - Attackers can identify the exact OS (e.g., Windows or Linux).
   - They can search for known vulnerabilities and exploits for that OS version.

2. Increased Risk of System Compromise

   - Knowing the OS makes it easier for attackers to exploit weaknesses and gain unauthorized access.

3. Service Disruption

- If attackers exploit the OS vulnerability, important services like websites, databases, or internal systems may go down.

3.

Vulnerability Name:-Nessus SYN Scanner

severity : Info

Plugin:-11219

Port :-80 / tcp / www

Description:-This plugin is a SYN 'half-open' port scanner. It shall be reasonably quick even against a firewalled target.Note that SYN scans are less intrusive than TCP (full connect) scans against broken services, but they might cause problems for less robust firewalls and also leave unclosed connections on the remote target, if the network is loaded.

Solution:-

1. Configure Firewall Rules

2. Close Unnecessary Ports

3. Use Intrusion Detection / Prevention Systems

Business Impact::-

1. Exposure of Network Services

- Attackers can discover open ports and active services.

- This reveals potential targets such as web servers, databases, or remote access services.

2. Increased Risk of Targeted Attacks

- Once services are identified, attackers can attempt exploits specific to those services.

3. Unauthorized Access

- If vulnerable services are exposed, attackers may gain unauthorized access to systems or applications.

4.

Vulnerability Name:-Device Type

severity : Info

Plugin:-54615

Description:-Based on the remote operating system, it is possible to determine what the remote system type is (eg: a printer, router, general-purpose computer, etc).

Solution:-

1. Limit Network Information Disclosure

2. Implement Strong Firewall Rules

3. Disable Unnecessary Services

Business Impact::-

1. Targeted Attacks

- Attackers can identify the type of device and search for known vulnerabilities related to that device.

2. Increased Risk of Network Breach

- Knowing device types helps attackers find weak points in the network infrastructure.

3. Unauthorized Access

- Vulnerable devices such as routers or servers may allow attackers to gain unauthorized control or access.

5.

Vulnerability Name:-OS Identification

severity : Info

Plugin:-11936

Description:-Using a combination of remote probes (e.g., TCP/IP, SMB, HTTP, NTP, SNMP, etc.), it is possible to guess the name of the remote operating system in use. It is also possible sometimes to guess the version of the operating system.

Solution:-

1. Configure Firewall Rules

2. Disable Unnecessary Services

3. Limit ICMP Responses

Business Impact::-

1. Targeted Cyber Attacks

- Attackers can identify the operating system and search for known vulnerabilities specific to that OS.

- This increases the chance of successful exploitation.

2. Unauthorized System Access

- If the OS has unpatched vulnerabilities, attackers may gain unauthorized access to systems.

3. Service Disruption

- Compromised systems can lead to downtime of important business services such as websites, applications, or internal platforms.

6.

Vulnerability Name:-Open SSL Detection

severity : Info

Plugin:-50845

Port :-443 / tcp / www

Description:-Based on its response to a TLS request with a specially crafted server name extension, it seems that the remote service is using the OpenSSL library to encrypt traffic.Note that this plugin can only detect OpenSSL implementations that have enabled support for TLS extensions (RFC 4366).

Solution:-

1. Update OpenSSL to the Latest Version

2. Apply System Security Patches

3. Disable Weak SSL/TLS Protocols

Business Impact::-

1. Exposure to Known Vulnerabilities

- Attackers can identify the OpenSSL version and search for public exploits affecting that version.

2. Data Breach Risk

- Vulnerabilities in OpenSSL may allow attackers to intercept encrypted communications, exposing sensitive data like passwords, financial information, or customer data.

3. Unauthorized System Access

- Certain OpenSSL vulnerabilities may allow attackers to execute malicious code or gain system access.

7.

Vulnerability Name:-Traceroute Information

severity : Info

Plugin:-10287

Port :-443 / tcp / www

Description:-Based on its response to a TLS request with a specially crafted server name extension, it seems that the remote service is using the OpenSSL library to encrypt traffic.

Solution:-

1. Block ICMP Responses

2. Configure Firewall Rules

3. Disable Unnecessary Network Responses

Business Impact::-

1. Network Reconnaissance

- Attackers can map the organization’s network infrastructure, including routers, gateways, and internal paths.

2. Identification of Critical Devices

- Revealed network paths may expose important network devices such as firewalls, routers, or servers.

3. Increased Risk of Targeted Attacks

- With knowledge of the network layout, attackers can plan targeted attacks against weak points in the infrastructure.

8.

Vulnerability Name:-Shor's Harvest Now Decrypt Later

severity : Info

Plugin:-298387

Description:-This algorithm easily computes two classically difficult mathematical problems used in modern cryptography; discrete logarithms, and factoring numbers formed by multiplying large primes. Shor's reduces both of these problems from taking exponential time in chosen cases to being solvable in polynomial time.

Solution:-

1. Implement Post-Quantum Cryptography

2. Use Hybrid Cryptography

3. Encrypt Sensitive Data with Strong Algorithms

Business Impact::-

1. Future Exposure of Sensitive Data

- Encrypted data stolen today (financial records, personal data, intellectual property) could be decrypted in the future when quantum computing becomes powerful enough.

2. Loss of Confidential Information

- Sensitive information such as customer data, trade secrets, and internal communications may be exposed.

3. Compliance and Regulatory Risks

- Organizations may face regulatory violations if protected data becomes readable later due to weak encryption.

Stage 3

Report

Title: Enforce least-privilege access through identity-aware, application-specific access policies using Zscaler Private Access (ZPA).

Zero Trust Network Access (ZTNA)
Zero Trust Network Access is a cybersecurity framework that shifts access control from a network-centric model to an identity-centric model. The foundational principle of ZTNA is "never trust, always verify," meaning that no user or device is trusted by default, regardless of whether they are connecting from inside or outside the corporate network. Unlike traditional remote access solutions that grant users broad access to the entire network, ZTNA creates a secure, application-specific tunnel between the user and the resources they are authorized to access. This ensures that the application remains invisible to unauthorized users and the broader internet, effectively shrinking the attack surface and eliminating the risk of lateral movement across the network.

Zscaler Private Access (ZPA)
Zscaler Private Access is a cloud-delivered ZTNA solution that provides identity- and contextbased access to private applications without granting network-level access. ZPA operates on a "serviceinitiated" model, where users connect to the Zscaler cloud, and the cloud brokers a secure connection to the specific application via ZPA App Connectors deployed in the data center or cloud environment. Because ZPA never places the user on the corporate network, internal IP addresses and network infrastructure are completely hidden. Access policies are dynamically enforced based on user

identity, group membership, device posture, and location, ensuring that each user sees only the applications they are explicitly permitted to use.

Principle of Least Privilege

The principle of least privilege (PoLP) is a security concept in which a user, device, or process is granted only the minimum levels of access—or permissions—necessary to perform its function. In the context of network security, this means moving away from "fat" network access (where authenticated users can reach large segments of the network) to "thin," application-specific access. By enforcing least privilege through ZPA, organizations ensure that even if a user's credentials are compromised, the attacker cannot pivot to other systems because the network itself is invisible and inaccessible. This principle is foundational to reducing insider threats and limiting blast radius during a security incident.

ZPA Architecture and Components

The ZPA architecture is composed of several key components that work together to provide secure, scalable access:

1. Zscaler Cloud (Service Edge): A globally distributed cloud platform that acts as the broker between users and applications. It handles authentication, policy enforcement, and connection brokering without sitting in the data path.

2. App Connectors: Lightweight software VMs or containers deployed within the private data center or cloud environments. These connectors initiate outbound connections to the Zscaler cloud, eliminating the need for inbound firewall rules. They establish secure tunnels to the ZPA edge and forward traffic to the internal applications.

3. ZPA Client Connector: A software client installed on end-user devices (Windows, macOS, Linux, iOS, Android). It authenticates the user, checks device compliance, and establishes a micro-tunnel to the ZPA edge for application access.

Implementation Methodology

The deployment of ZPA follows a structured, phased approach to ensure minimal disruption and maximum security efficacy:

• **Phase 1:** Discovery and Inventory: Identify all internal applications currently in use. Categorize them by criticality, department, and user base. Tools like network scanners, configuration management databases (CMDB), and interviews with department heads are used to build a comprehensive application inventory.

• **Phase 2:** Connector Deployment: Deploy ZPA App Connectors in the data center or cloud environments where the applications reside. This involves provisioning virtual machines (using VMware, Hyper-V, or cloud-specific images) and configuring them to connect to the ZPA cloud.

• **Phase 3:** Application Onboarding: Define application segments within the ZPA console. This includes specifying the IP addresses and ports of the internal applications and associating them with the appropriate App Connector groups.

• **Phase 4:** Policy Creation: Integrate ZPA with the corporate identity provider (e.g., Azure AD). Create granular access policies based on user groups, device posture, and location. Policies are tested in a simulation mode before enforcement.

Tools and Technologies

• **Primary Platform:** Zscaler Private Access (ZPA) – The core Zero Trust access broker.

• **Identity Provider (IdP):** Microsoft Azure Active Directory (AD), Okta, or Ping Identity for user authentication and group synchronization.

• **Endpoint Protection Platform (EPP):** Integration with endpoint security tools (e.g., CrowdStrike, SentinelOne, Microsoft Defender for Endpoint) to assess device posture and hygiene before granting access.

• **Security Information and Event Management (SIEM):** Integration with SIEM platforms (e.g., IBM QRadar, Splunk, LogRhythm) for centralized logging and correlation of access events.

• **Virtualization Platforms:** VMware vSphere, Microsoft Hyper-V, or public cloud environments (AWS, Azure, GCP) for hosting ZPA App Connectors.
• **Infrastructure as Code (IaC):** Tools like Terraform or Ansible for automated deployment and configuration of App Connectors at scale

Integration with Identity Providers

Integrating ZPA with the organization's existing identity provider is a critical step in establishing identity-aware access. This integration is typically done via SAML (Security Assertion Markup Language) or LDAP (Lightweight Directory Access Protocol). Once integrated, ZPA synchronizes user attributes and group memberships, allowing policies to reference these groups directly. For example, when a user authenticates via the ZPA Client Connector, they are redirected to the corporate IdP for credentials and multi-factor authentication (MFA). Upon successful authentication, ZPA receives identity attributes that are then used to evaluate access policies and determine which applications the user is allowed to see and connect to.

Device Posture Checking

Beyond user identity, ZPA can enforce policies based on the security posture of the connecting device. Through integration with endpoint detection and response (EDR) and mobile device management (MDM) solutions, ZPA can assess factors such as:
• Antivirus software status and definition updates
• Disk encryption status (e.g., BitLocker, FileVault)
• Operating system patch level
• Presence of jailbreak or root compromise on mobile devices
• Real-time risk score from the EDR tool
If a device fails to meet the defined posture requirements (e.g., antivirus is disabled), ZPA can block access, grant limited access, or trigger a remediation workflow.

Incident Response in a ZPA Environment

ZPA significantly streamlines the incident response process. When a potential threat is detected, security teams can leverage ZPA's capabilities to:

1. Isolation: Immediately revoke access for a compromised user account or device through the ZPA console, cutting off their connectivity to all internal applications.

2. Forensics: Access detailed logs within ZPA showing exactly which applications were accessed, by whom, from which location, and for how long.

3. Containment: Because users have no network-level access, a compromised endpoint cannot be used to scan or attack other internal systems, containing the blast radius.

4. Granular Blocking: Temporarily block access to specific high-risk applications while allowing access to others, ensuring business continuity during an investigation.

Understanding ZPA: Key Takeaways

Zscaler Private Access represents a fundamental shift in how remote access is architected and secured. Here are the key aspects to understand about the solution:

1. It is cloud-native: ZPA is delivered entirely from the Zscaler cloud, eliminating the need for hardware appliances and complex VPN head-end management.

2. It is identity-centric: Policies are built around users and groups, not IP addresses, making access control more intuitive and aligned with organizational structure.

3. It ensures network invisibility: By never placing users on the network, ZPA renders internal infrastructure invisible to the internet, preventing reconnaissance and DDoS attacks.

4. It is scalable: The cloud architecture allows ZPA to scale instantly to accommodate thousands of concurrent users without performance degradation.

5. It supports hybrid environments: ZPA works seamlessly for applications hosted in on-premises data centers, private clouds, and public cloud environments.

Conclusion:

Stage 1: In conclusion, web application testing is a crucial process that ensures the security, functionality, and reliability of web-based software. It involves a comprehensive evaluation of various aspects, including user interface, functionality, performance, and security to identify and rectify issues before they impact end-users. This process not only enhances the quality of web applications but also bolsters their resilience against potential threats and vulnerabilities. In an age where online services and applications are integral to our daily lives, effective web application testing is essential to deliver a seamless and secure user experience.

Stage 2: The Nessus report for the specific website has highlighted key security insights. It identified vulnerabilities, categorized by severity, with a focus on critical and high-risk issues that need immediate attention. Recommendations for addressing these vulnerabilities include patching, reconfiguration, and other security measures, with a priority on actions that can have the most significant impact. Compliance with industry standards and best practices is considered, ensuring alignment with recognized security benchmarks like OWASP guidelines. The report underscores the need for continuous monitoring, regular updates, and ongoing security efforts to maintain a strong security posture. In summary, the Nessus report offers a clear view of the website's security state, provides guidance on mitigating vulnerabilities, and stresses the importance of sustained vigilance against evolving threats.

Stage 3: The implementation of Zscaler Private Access marks a pivotal shift from a vulnerable, networkcentric security model to a resilient, identity-centric Zero Trust architecture. By enforcing leastprivilege access and rendering private applications invisible to the internet, this project

effectively neutralizes the threats of lateral movement and network-based attacks. The solution not only fortifies the organization's security posture against modern cyber threats but also delivers a seamless and high-performance experience for a distributed workforce. Furthermore, the decoupling of access from the network simplifies IT operations, reduces dependency on legacy VPN infrastructure, and provides granular visibility into user-application interactions. Ultimately, this initiative establishes a future-ready foundation for secure digital transformation..

Future Scope:

Stage 1: future scope of web application testing

The future scope of web application testing is promising and evolving rapidly, driven by the continuous growth of the internet and web-based applications.

Here are some key aspects of its future scope:

▪ Increased Demand: As more businesses and services move online, the demand for web applications continues to grow. This, in turn, increases the demand for web application testing to ensure security, reliability, and functionality.

▪ Advanced Technologies: The complexity of web applications is rising with the integration of advanced technologies like AI, IoT, and blockchain. Testing these cutting-edge applications will require specialized skills and tools.

▪ Security Emphasis: With the growing number of cyber threats and data breaches, security testing for web applications will become even more critical. Testers will need to focus on vulnerabilities, data protection, and compliance with regulations like GDPR.

▪ Mobile and Cross-Browser Testing: Mobile usage is on the rise, and web applications must be responsive and functional on various devices and browsers. Testing for cross-compatibility will be essential.

▪ Performance Testing: As user expectations for speed and performance increase, load testing, stress testing, and other performance testing types will be crucial to ensure web applications can handle traffic spikes.

Stage 2: future scope of the testing process you understood.
The future scope of the web application testing process, particularly in terms of identifying vulnerabilities, is exceedingly promising due to the continuous growth of digital platforms, the escalating importance of cybersecurity, and the ever-evolving threat landscape. Here are some key aspects that highlight the prospective scope of web application testing for vulnerability discovery:
▪ AI-Driven Testing: The integration of artificial intelligence (AI) and machine learning (ML) into web application testing tools will revolutionize the way vulnerabilities are identified. AI algorithms can swiftly scan vast amounts of code and data to pinpoint vulnerabilities that might elude human testers.
▪ IoT and Mobile App Testing: With the proliferation of Internet of Things (IoT) devices and the increasing usage of mobile applications, the scope of web application testing will extend to encompass these areas. Ensuring the security of IoT devices and the interfaces they use will become critical.
▪ Blockchain and Cryptocurrency Security: As blockchain technology and cryptocurrencies continue to gain traction, security testing will be imperative to safeguard transactions and digital assets. Ensuring the robustness of blockchain-based applications will be a specialized niche in web application testing. Serverless Architectures: With the adoption of serverless computing, web application testing will need to adapt to assess the security of serverless functions and APIs. Ensuring that serverless components are free from vulnerabilities will be paramount.

Stage 3: future scope of Zero Trust
The successful deployment of ZPA lays the groundwork for a broader, more integrated security ecosystem. Future enhancements can be explored to further mature the organization's security capabilities:
▪ Integration with Zscaler Internet Access (ZIA): Converging Private Access (ZPA) with Internet Access (ZIA) to create a unified Secure Access Service Edge (SASE) platform, managing all web and private traffic through a single policy framework.

▪ Advanced Device Posture Checking: Moving beyond basic compliance to incorporate real-time risk scores from EDR tools, blocking access if a device shows signs of active compromise.
▪ Privileged Remote Access: Extending ZPA to third-party vendors and contractors with time-bound, audited access to specific critical systems, replacing jump boxes and bastion hosts.
▪ IoT/OT Segmentation: Applying the same micro-tunneling principles to Internet of Things (IoT) and Operational Technology (OT) devices to ensure they can only communicate with specific management servers.
▪ AI-Driven Anomaly Detection: Leveraging machine learning on ZPA logs to detect behavioral anomalies, such as a user accessing an application at an unusual hour or from a strange location, triggering automated MFA challenges or access revocation.
▪ Cloud Workload Protection: Utilizing ZPA to secure connections between containers and microservices in multi-cloud environments (AWS, Azure, GCP), ensuring east-west traffic between workloads is encrypted and authenticated.
▪ Automated Provisioning/Deprovisioning: Enhancing the identity provider integration to automatically revoke application access the moment an employee is terminated or changes roles, ensuring near real-time access control.
▪ Digital Experience Monitoring (DEM): Implementing DEM tools to proactively monitor the enduser experience, identifying network latency or application performance issues before they impact productivity.

Topics Explored:
1. Zscaler
2. Zero trust
3. ZPA
4. VPN
5. MISP
6. College Network Information

7. Deployment of SOC in the rkCollege
8. Threat Intelligence
9. Incident Response

Tools Explored:

- Nessus Scanner